3 DRAW x,y,angle: circular arcs

Kind: improvement, with one new option.

3.1 Overview

The new ROM replaces the routine behind the three-parameter form DRAW x,y,angle, which draws a circular arc from the current graphics position to the point offset by (dx, dy) while turning through a signed angle A. The original 1982 routine is an exceptional piece of engineering for its time. Given only the chord between the start and end points and a signed sweep angle, it reconstructs the required circular geometry, shares its implementation with CIRCLE, and preserves the command's exact endpoint behavior. The cost of that generality is substantial: the routine occupies 291 bytes of ROM and relies on the software floating-point calculator for a square root and several trigonometric evaluations on each invocation. The new implementation performs the same externally visible operation in 218 bytes, freeing 73 bytes that pay for the faster CIRCLE and PLOT routines (Chapters 4 and 5).

Unlike CIRCLE, where the caller supplies the center and radius, DRAW x,y,angle supplies neither. The center must therefore be reconstructed from the chord and the requested sweep angle. At the same time, the routine has a strict interface contract: whatever approximations are made internally, the graphics cursor must finish on the exact destination pixel. All of this must be achieved on a Z80 with no hardware multiply or divide instructions and no hardware floating-point unit. Operations delegated to the Spectrum's software calculator can cost hundreds of thousands of T-states, making the choice of numerical representation as important as the geometry itself.

The central design principle is therefore simple:

Precision is a budget imposed by the screen, and it should be spent only where the screen can reveal it.

The Spectrum display contains only 256×176 addressable pixels. After a geometric coordinate is rounded to the raster grid, numerical refinement that does not change the selected pixel has no observable benefit. A curve whose maximum geometric deviation is a tenth of a pixel, for example, cannot be distinguished from the ideal curve once both are rasterized. Spending processor time to compute that tenth of a pixel more accurately is therefore wasted precision.

The endpoint is the deliberate exception it is part of the command's contract.

That principle determines the design of the new routine. The center is found from a single half-angle coefficient, with no square root (*Locating the center without a square root*). Repeated trigonometry is replaced by a fixed incremental rotation (*Tracing the arc*). The number of segments is derived from a sagitta requirement and then approximated by a straight line that is cheap on the Z80 (*Choosing the number of segments*). Independently rounded segments are prevented from accumulating endpoint error (*Closed-loop rasterization*). Finally, the limits are handled explicitly, including multi-turn sweeps (*Sweeps beyond one turn*).

The result is therefore more than a shorter implementation of the original algorithm. It is a deliberate reallocation of numerical effort: precision is removed where the display cannot reveal it and preserved where the ROM interface requires it.

3.1.1 What is new for the programmer

- **Multiples of 2π draw a straight line.** The 1982 routine raised report B, "Integer out of range", on 21 of the first 60 multiples of 2π; the new routine draws the straight line for all 60 (see *Multiples of 2π*).
- **Optional angle reduction.** Setting bit 6 of FLAGS2 makes DRAW replace the angle by its remainder modulo 2π: `POKE 23658,PEEK 23658+64` turns it on and `POKE 23658,PEEK 23658-64` turns it off. It is off by default, so existing programs behave exactly as before (see *Angle reduction*).

- **Fewer arcs miss the ideal circle by more than a pixel:** 4.2% of 800 random arcs, against 12.0% for the 1982 routine, at about 5% more time over the timing grid.

3.2 What the command must guarantee

DRAW x,y draws a straight line to the point offset by (dx,dy); DRAW x,y,angle draws the circular arc between the same endpoints while turning through A radians [1]. Programs can chain several DRAW operations to construct closed shapes — rounded rectangles, gears, dials and other outlines — so an endpoint error that is barely visible in one arc can become obvious when many arcs are joined. The destination pixel must be exact within the constraints of the machine.

3.2.1 Hardware and arithmetic constraints

- Display: 256×176 addressable pixels, with BASIC coordinates 0–255 horizontally and 0–175 vertically; one bit per pixel.
- CPU: Z80 at 3.5 MHz, with an 8-bit ALU and 16-bit addition/subtraction, but no hardware multiply, divide, or floating-point unit.
- Floating point: the ROM software calculator (the FP-CALC stack machine) maintains its own stack, separate from the Z80 hardware stack. SIN, COS, multiplication and division are all calculator operations.

Fitting the available space

The replacement has to fit between the existing circle code and DRAW-LINE at $24B7, whose address is referenced elsewhere in the ROM and therefore cannot move. The 1982 arc code occupies 291 bytes. The new implementation occupies 218 bytes. Those 218 bytes include both the multiple-of-2π guard and the optional angle-reduction mechanism described under *Sweeps beyond one turn*.

3.3 Locating the center without a square root

The first design decision concerns the circle center. The 1982 routine does not need to form the center explicitly; it scales and rotates the chord to obtain an initial step vector. The new implementation takes a different route and reconstructs the center directly. The construction costs only one half-angle coefficient and no square root. In return, it produces the radius vector from the center to the start point — precisely the vector needed by the incremental rotation described in the next section and, through its components, a convenient measure of the radius for the segment count.

3.3.1 The problem in one picture

Let the start point be P_0, let the chord vector be (dx, dy), and let $P_1 = P_0 + (dx, dy)$ be the endpoint. The routine sweeps a signed angle A about an unknown center C. A textbook construction would bisect the chord and then move along its perpendicular bisector by a distance derived from the radius using Pythagoras. That requires a square root and, because the radius is not known, an additional division — both costly operations here. Figure 3.1 uses a more convenient construction based on the half-angle $\frac{A}{2}$ at the center.

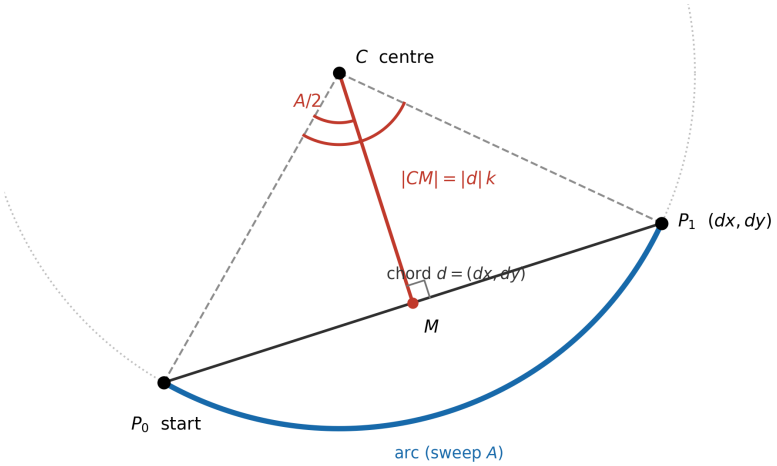


Figure 3.1 — Geometry used to locate the center. The chord d = (dx,dy) subtends the signed angle A at C; M is its midpoint, CM is perpendicular to the chord, and |CM| is expressed directly through the half-angle coefficient.|d|·k k = ½·cot(A/2)

3.3.2 A purely 2D derivation

Using only plane Euclidean geometry. Let the start point be $P_0 = (0,0)$ and let the chord vector be d = (dx, dy), and P_1 = $P_0 + d$. Let C = (C_x, C_y) be the unknown center and let M be the midpoint of the chord P_0P_1.

$$M = \left(\frac{dx}{2}, \frac{dy}{2}\right)$$

For any chord of a circle, the radius drawn to the chord's midpoint is perpendicular to the chord, and it bisects the central angle A. So CM is perpendicular to P_0P_1, and the angle P_0CM equals $\frac{A}{2}$.

Consider the right triangle CMP_0, the two legs adjacent to that half-angle are the half-chord MP_0 = $\frac{|d|}{2}$ and the segment CM itself, giving

$$\cos\left(\frac{A}{2}\right) = \frac{CM}{CP_0}$$

$$\sin\left(\frac{A}{2}\right) = \frac{MP_0}{CP_0}$$

Dividing the two equations eliminates the unknown radius CP_0 entirely:

$$\frac{\cos\left(\frac{A}{2}\right)}{\sin\left(\frac{A}{2}\right)} = \frac{CM}{MP_0}$$

$$CM = MP_0 \cdot \cot\left(\frac{A}{2}\right)$$

Since MP_0 is half the chord length:

$$CM = \frac{|d|}{2}\cot\left(\frac{A}{2}\right)$$

Define the single half-angle coefficient:

$$k = \frac{1}{2} \bullet cot\frac{A}{2}$$

$$CM = |d| \cdot k,$$

The important simplification is that the length $|d|$ does not actually have to be calculated.

The chord vector is $d = (dx, dy)$. A perpendicular vector with the same magnitude is:

$$n = (-dy, dx)$$

because:

$$|n| = \sqrt{-dy^2 + dx^2} = |d|$$

Consequently, the vector from M to C can be written directly as the perpendicular vector scaled by k:

$$CM = (-dy \cdot k,\ dx \cdot k)$$

No normalization is required. The factor $|d|$ that would normally appear when constructing a unit perpendicular vector is exactly canceled by the $|d|$ present in CM.

Finally:

$$C = M + CM$$

Substituting M and CM:

$$C = \left(\frac{dx}{2}, \frac{dy}{2}\right) + (-dy \cdot k,\ dx \cdot k)$$

Therefore:

$$C_x = \frac{dx}{2} - dy \cdot k$$

$$C_y = \frac{dy}{2} + dx \cdot k$$

Why this construction is useful

The cancellation of $|d|$ is the key observation. The geometry therefore maps directly onto a low-cost implementation: the center is calculated from the chord components and one half-angle coefficient, without a square root and without explicitly calculating the radius.

Only one division and one evaluation each of SIN and COS, both at the single argument $\frac{A}{2}$, are needed to obtain the center. Crucially, that cost is paid once per arc, not once per segment. The vector from the center to the start point, $R_o = -C$, is the vector rotated incrementally.

3.4 Tracing the arc

Incremental rotation instead of repeated trigonometry.

3.4.1 The recurrence

Once the center is known, the circular path is approximated by N straight segments. N must be large enough that the polygon is visually indistinguishable from the circle, but not so large that the extra work exceeds what the pixel grid can reveal. A straightforward implementation could evaluate SIN and COS at every point on the arc. The new routine instead evaluates them once for the fixed per-segment angle δ and reuses the resulting pair throughout the loop

$$Rx' = Rx \cdot \cos\delta - Ry \cdot \sin\delta \quad Ry' = Rx \cdot \sin\delta + Ry \cdot \cos\delta$$

Starting from the radius vector R_0 obtained as described above, each recurrence advances the vector by exactly one segment angle. After N applications, the accumulated rotation is A by construction. Figure 3.2

shows the resulting polygon for a representative value of N: every radius vector $R_0 \ldots R_N$ is obtained from its predecessor by the same fixed rotation, with no new absolute-angle trigonometry R_0 = -C.

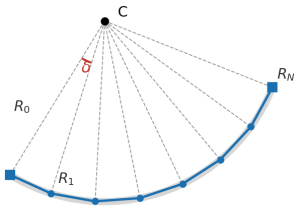


Figure 3.2 — Incremental rotation. The radius vector advances by the same fixed angle δ at every step; cos δ and sin δ are evaluated once and reused for all N segments.

3.4.2 The 1982 routine

The original Sinclair DRAW x,y,angle routine already uses incremental rotation. It derives an initial segment vector from the complete chord (dx, dy), using a scale factor and an initial rotation to orient the vector correctly. That vector is then rotated by the same fixed angle at every iteration. No explicit circle center or radius is required.

The first chord is scaled by $W = \frac{\sin\left(\frac{A}{2N}\right)}{\sin\left(\frac{A}{2}\right)}$ and rotated through $\frac{\left(A-\frac{A}{N}\right)}{2}$.

The resulting chord vector is then rotated by $\frac{A}{N}$ at every step.

The new implementation takes a different route. It first reconstructs the circle center from the chord and swept angle and then forms the radius vector from the center to the start point:

$R_o = P_o - C$

The same fixed-angle recurrence is then applied to this radius vector.

3.5 Choosing the number of segments

The number of segments N determines the angular increment used by the rotation recurrence:

$$\delta = \frac{A}{N}$$

Increasing N produces a finer polygonal approximation but also increases the number of operations in the tracing loop. The useful value of N is therefore determined jointly by computational cost and by the resolution of the 256×176 display.

The 1982 routine [2] chooses N from both the circle size and the sweep. It forms

$$Z = \left| \frac{|X| + |Y|}{\sin\left(\frac{A}{2}\right)} \right|$$

a conservative over-estimate of the diameter, and uses

$$N_{1982} = 4 \cdot INT\left(\frac{INT\left(|A|\sqrt{Z} + \frac{1}{2}\right)}{8} \right) + 4 \leq 252$$

The new N

For a chosen sagitta s,

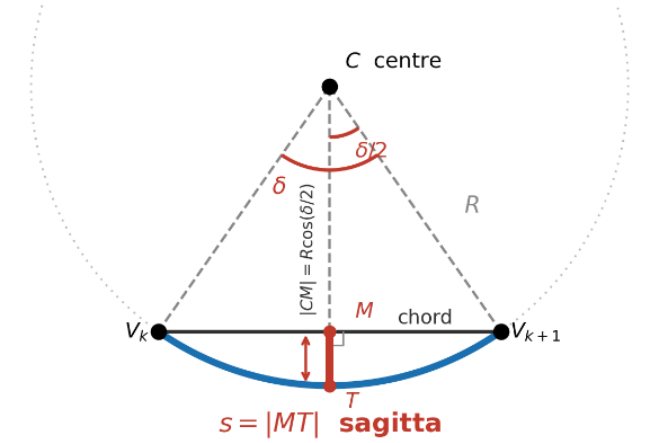


Figure 3.3 — sagitta $s = |MT|$

$$s = R\left(1 - \cos\left(\frac{\delta}{2}\right)\right) \approx \frac{R\delta^2}{8}$$

The segment count follows the familiar relationship $N = |A|\sqrt{\frac{R}{8s}}$.

Consider the large circle in Figure 3.4, with R = 76 pixels and a sweep of 5.4 rad. The 1982 routine uses 40 segments, giving a sagitta of 0.17 pixel. A deliberately coarse 22-segment polygon has a sagitta of 0.57 pixel and is visibly flat at its worst point.

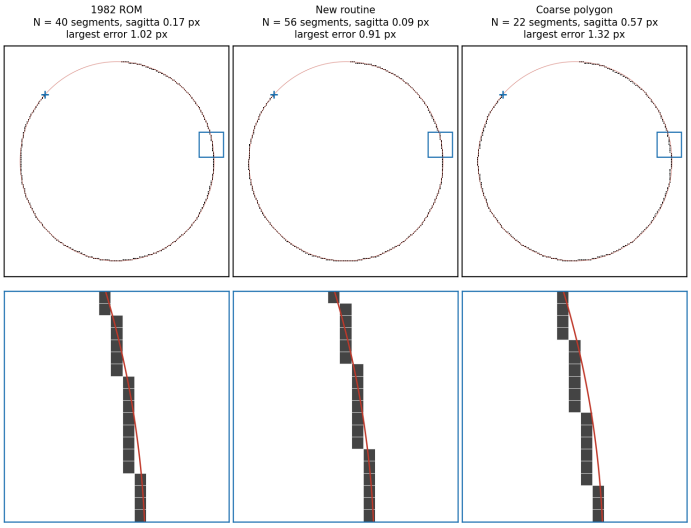


Figure 3.4 — The same arc (radius 76 pixels, sweep 5.4 rad) drawn by the 1982 ROM (left) and by a deliberately coarse polygon (right). Panel titles give the segment count, sagitta and maximum distance of any lit pixel from the ideal circle. The bottom row enlarges the 18×18-pixel region marked in the top row, where the coarse polygon deviates most strongly.

Its largest pixel error is 1.32 pixels, compared with 1.02 pixels for the 1982 routine. The measurements suggest that a sagitta of roughly one tenth of a pixel keeps almost all arcs within one pixel of the ideal circle, whereas a sagitta near half a pixel does not. That becomes the practical precision budget.

Practical implementation.

$$N = |A|\sqrt{\frac{R}{8s}}$$

The exact radius is not available without a square root, but the new construction provides the radius vector $R_0 = (R_{x0}, R_{y0})$ from the center to the start point. The quantity

$$R' = |R_{x0}| + |R_{y0}|$$

lies between R and $\sqrt{2}\cdot R$ and has mean value $\frac{4R}{\pi} \approx 1.27\ R$ for uniformly distributed directions. Thus, using $R \approx \frac{R'}{1.27}$ and $s = 0.1$ pixel, the ideal segment-density law becomes $\frac{N}{A} \approx 0.99\ \sqrt{R'}$.

On the 256×176 display, a circle drawn entirely on screen has $R \leq 87$ pixels, giving $R' \leq 125$; larger circles are only partially visible. The useful operating range begins at approximately $R' = 10$, since for smaller values pixel quantization dominates the geometric approximation error.

Over the practical range $10 \leq R' \leq 125$, the square-root term is closely approximated by the simple affine expression $c + R'/16$ requiring only a shift and an addition. The approximation slightly overestimates the smallest circles and gradually converges towards $\sqrt{R'}$for larger radii, remaining within about 16% throughout the useful range.

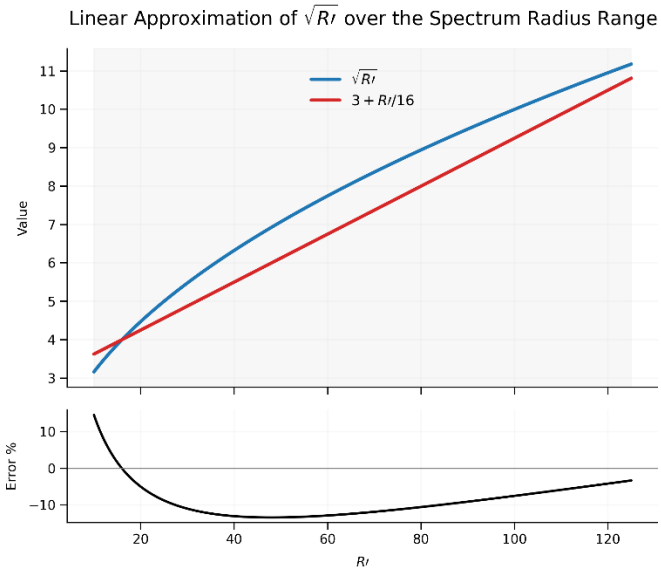


Figure 3.5 — R′ vs 3 + R′/16

R′ (px)	Ideal	3 + R′/16	Error
10	3.13	3.62	+16%
20	4.43	4.25	−4%
40	6.27	5.50	−12%
60	7.67	6.75	−12%
80	8.86	8.00	−10%
100	9.91	9.25	−7%
125	11.08	10.81	−2%

Table 3.1 — Segments per radian required by the ideal law for a sagitta of 0.1 pixel (0.99·√R′), and by the straight line 3 + R′/16, with the error of each. R′ from 10 to 125 covers the circles that fit on the screen.

We therefore use the following approximation for the required segment count:

$$N = 4 + ROUND\left(|\ A\ |\ \ (c + R'/16)\right)$$

The additive term of 4 establishes a sensible minimum segment count, preventing small arcs from collapsing into overly coarse approximations consisting of only one or two chords.

Choosing the constant c empirically. The analytical derivation points to a constant close to 3.5, but the underlying sagitta criterion is itself approximate. Rather than relying solely on the theoretical fit, the constant was tuned experimentally. Three hundred random arcs were generated, spanning radii from 8 to 85 pixels and sweep angles from 0.4 to 5.6 radians, with random start angles and directions. Each test case was rendered by a set of otherwise identical implementations differing only in the value of the constant. Performance was measured in Z80 T-states, while rendering accuracy was assessed by comparing the resulting bitmap with the ideal circular arc. The error metric was the maximum radial deviation of any illuminated pixel from the corresponding ideal circle.

Law	Time (1982 = 1.00)	Arcs > 1 px	By radius (< 25 / 25–50 / 50–70 / ≥ 70)
1982 ROM	1.00	10%	3% / 7% / 14% / 20%
c = 1	0.90	11%	7% / 12% / 12% / 12%
c = 2	1.01	7%	0% / 9% / 10% / 8%
c = 2.5	1.06	4%	0% / 3% / 6% / 10%
c = 3 (adopted)	**1.11**	**2%**	**0% / 4% / 3% / 0%**
c = 3.5	1.16	4%	0% / 3% / 6% / 8%
c = 4	1.21	1%	0% / 2% / 1% / 0%

Table 3.2 — The constant c of the law N = 4 + ROUND(|A|·(c + R′/16)) over the same 300 arcs. Times are relative to the 1982 routine; the last column gives the share of arcs beyond one pixel for circles of radius below 25, 25–50, 50–70 and 70 pixels or more.

Combining the geometric approximation, the linearized square-root term, and the empirically determined constant yields the final expression:

$$N = 4 + ROUND\left(|A| \cdot \frac{R' + 48}{16}\right)$$

With c = 3 selected, the final routine was evaluated again on 800 new random arcs using a different random seed. The fraction of arcs more than one pixel from the ideal circle falls from 12.0% for the 1982 routine to 4.2% for the new routine, with an improvement in every radius class. The mean maximum error changes only modestly, from 0.835 to 0.806 pixel; the principal gain is therefore in the tail — the worst arcs — rather than in the typical case.

Radius (px)	Arcs	> 1 px 1982	> 1 px new	Mean 1982	Mean new	P95 1982	P95 new
< 25	175	5.7%	0.6%	0.731	0.699	1.01	0.94
25–50	267	8.2%	3.4%	0.830	0.806	1.03	0.98
50–70	209	17.7%	9.1%	0.881	0.854	1.06	1.04
≥ 70	149	18.1%	3.4%	0.900	0.867	1.06	1.00
All	**800**	**12.0%**	**4.2%**	**0.835**	**0.806**	**1.05**	**1.00**

Table 3.3 — The 1982 routine and the new routine over 800 new random arcs: share of arcs more than 1 px from the ideal circle, and mean and 95th percentile of the largest radial error of each arc, by radius of the circle.

Swept angle |A|	Arcs	1982 ROM	New routine
< 1 rad	16	1.00	0.94
1–2 rad	16	1.00	1.00
2–4 rad	16	1.00	1.06
> 4 rad	11	1.00	1.18

Swept angle |A|	Arcs	1982 ROM	New routine
All	**59**	**1.00**	**1.05**

Table 3.4 — Time relative to the 1982 routine (= 1.00), by swept angle.

Across the grid, the new routine is faster for short sweeps and slower for the longest ones, where the improved segment law requests more chords. Over the 59-arc timing grid it takes 5% more time than the 1982 implementation. (These times were measured before the faster multiplication of Chapter 18. An arc makes about 90 floating-point multiplications, so the new routine is now faster than the figures of Table 3.4 show; the 1982 column is the original ROM and does not change.)

The linear law therefore emerges as the preferred solution, delivering near-optimal visual fidelity while preserving the speed, compactness and characteristic behavior of the original 1982 routine.

3.6 Closed-loop rasterization

The recurrence described above is exact in real-number arithmetic, but every generated point must eventually become an integer pixel displacement for DRAW-LINE. A naive implementation rounds each segment independently and then draws it. Because the rounding error of one segment is unrelated to the next, these errors can accumulate rather than cancel. Over many segments, the final cursor can therefore drift away from the requested endpoint — especially for short chords swept through large angles, where the geometry is fine relative to the coarseness of the pixel grid.

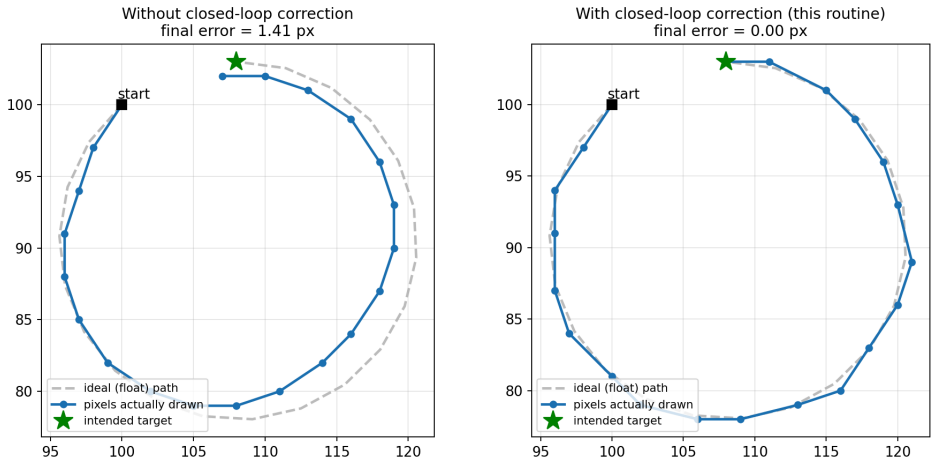


Figure 3.6 — Endpoint drift caused by independent per-step rounding. The test arc uses chord (8,3), a 320° sweep and 20 segments. The dashed curve is the ideal floating-point path; the closed-loop implementation reaches the requested destination exactly.

The original ROM DRAW routine already contains the essential mechanism: each successive polyline segment begins at the graphics cursor's actual rasterized position, read through COORDS, rather than at an independently accumulated ideal position. Any rounding error introduced by one segment is therefore absorbed into the starting point of the next. The new arc routine adopts the same discipline while making the two positions explicit. At each step, the ideal point is advanced in floating point, the actual cursor is read from COORDS, and only the difference between those positions is passed to DRAW-LINE, which performs the final pixel rounding.

Concretely, the ideal point is not recomputed from a stored center plus the current rotated vector; it is accumulated directly, by adding the vector's step-to-step difference:

$$\text{ideal}' = \text{ideal} + (R' - R)$$

This update is mathematically equivalent to maintaining ideal = center + R at every step, but it has an important implementation advantage: the center no longer needs to be stored. It cancels from the step-to-

step difference, allowing the running ideal point to remain entirely on the calculator stack. The ordinary-arc implementation is therefore complete. The next section turns to the exceptional case of multi-turn sweeps, where the one-byte segment count and the command's unrestricted angle meet the limits of the arithmetic.

3.7 Sweeps beyond one turn

The sections above considered ordinary sweeps. DRAW, however, accepts an arbitrary angle, and very large angles expose a different set of limits. The segment count is stored in a byte, while the 1982 routine does not reduce the supplied angle modulo 2π. The polygon can therefore wind around the same circle many times.

In May 1983 Ian Logan, co-author of the standard annotated ROM disassembly, explained in Popular Computing Weekly [3] why a command such as PLOT 50,50: DRAW 50,50,9999 produces a striking pattern rather than an ordinary arc (*Figure 3.7*, top row). His diagnosis was that the angle is not reduced modulo 2π and that the segment count saturates at 252. The first segment is therefore far too long, while the remaining 251 segments continue trying to reach the destination.

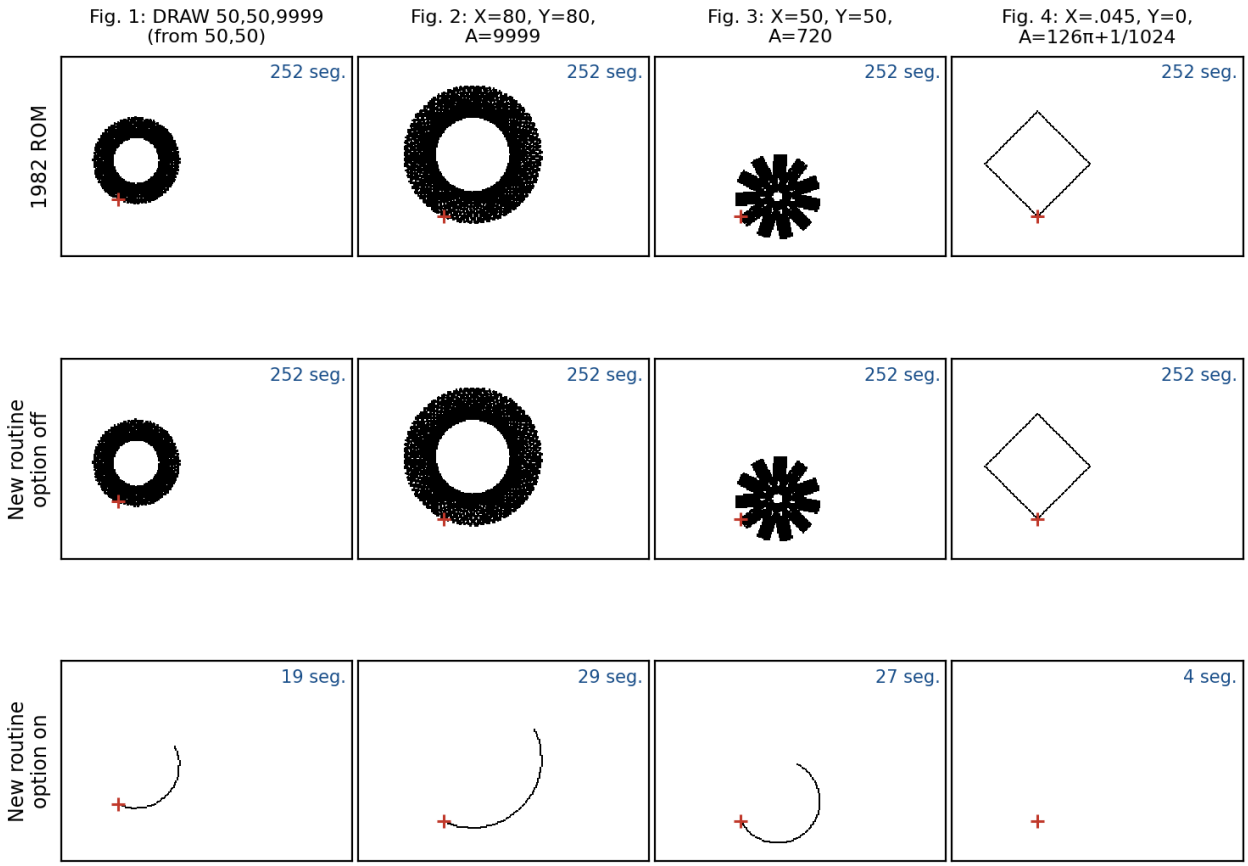


Figure 3.7 — Logan's four examples (columns: Fig. 1, DRAW 50,50,9999 from (50,50); Fig. 2, X = 80, Y = 80, A = 9999; Fig. 3, X = 50, Y = 50, A = 720; Fig. 4, X = 0.045, Y = 0, A = 126π + 1/1024; the last three from (75,35), as in his program). The red cross marks the start point.

Top: 1982 ROM. Middle: new routine with angle reduction disabled; it produces the same exact pixels as the 1982 ROM. Bottom: new routine with reduction enabled; the angle is replaced by its signed remainder modulo 2π (see *Angle reduction* below), producing the ordinary arcs suggested by Logan's corrected input.

3.7.1 Saturation

For ordinary arcs, the cap of 252 is rarely relevant; it is reached systematically in Logan's large-angle examples. The new routine uses the same limit. Logan's four examples are reproduced pixel for pixel (Jaccard similarity 1.000). Small circles with very large angles also matched in all 12 tested cases. Alternative caps of 64 and 128 were tested but produced different patterns — thin rings or nearly filled discs — with low similarity (0.01–0.60) and no useful visual advantage. The value 252 was therefore retained for drop-in behavioral compatibility.

3.7.2 Angle reduction

Reducing the angle modulo 2π is mathematically valid for the geometric circle defined by DRAW: the endpoint is fixed, and the circle center and radius depend on A through cot(A/2), which is 2π-periodic. This is the substance of Logan's observation [3]. The present implementation exposes the same idea as an optional run-time feature: A is replaced by A − 2π·TRUNC(A/2π), preserving the sign of the supplied angle.

The reduction changes the path, not merely its numerical representation. A sweep of 7 rad comprises one complete turn plus a further 0.72 rad; the original ROM draws the complete multi-turn path, whereas the remainder draws only the final 0.72-rad arc. In Logan's A = 9999 example, the remainder is about 2.45 rad (140°), producing an ordinary arc instead of the characteristic ring. The extra turns of any sweep greater than 2π are likewise discarded. For that reason the option is disabled by default: it is a useful compatibility extension, not a silent change to DRAW semantics.

The reduction is selected at run time by bit 6 of FLAGS2, the system variable at 23658 ($5C6A). The arc setup tests BIT 6,(IY+$30) and, when set, replaces A by its signed remainder modulo 2π. Because Sinclair BASIC AND and OR are logical rather than bitwise operators, the flag is enabled by adding 64 to FLAGS2 and disabled by subtracting 64. It is clear after NEW and at power-on, where the RAM test initializes the system variable to $02.

The following BASIC program draws the same arc without and with the reduction, side by side:

```
10 CLS : PLOT 50,50: DRAW 50,50,9999
20 POKE 23658,PEEK 23658+64
30 PLOT 150,50: DRAW 50,50,9999: POKE 23658,PEEK 23658-64
```

3.7.3 Multiples of 2π

The new DRAW routine retains the essential straight-line test of the original implementation: immediately after $\sin\left(\frac{A}{2}\right)$ is computed, its magnitude is tested and, when sufficiently small, the trigonometric results are discarded and it draws the straight chord. This also covers $A = 0$, so the separate zero-angle entry test of the original routine is no longer required.

An exact floating-point comparison is unreliable, however, because $\sin\left(\frac{A}{2}\right)$ is not represented as exactly zero for many exact multiples of 2π. **The original ROM routine consequently fails** on 21 of the first 60 tested multiples, raising Report B, 'Integer out of range'. The new implementation treats

$\left|\sin\left(\frac{A}{2}\right)\right| < 2^{-14}$ as zero.

At that threshold the implied radius exceeds 8000 times the chord, so the rasterized result is indistinguishable from the straight line. The failure of the 1982 routine can be reproduced on real hardware with the following three-line BASIC test. Each iteration draws from (128,88) to (148,98); using an angle of A = $2k\pi$. Since each angle is an integer multiple of a full revolution, the geometrically correct result is always the straight line between the two endpoints.

```
10 CLS : LET n=0
20 LET n=n+1: PRINT AT 0,0;"k=";n;" ": PLOT 128,88: DRAW 20,10,2*PI*n
30 IF n<60 THEN GO TO 20
```

On the original ROM, this program fails in 21 of the 60 iterations (k = 9, 11, 18, 19, 21, 22, 23, 31, 36, 37, 38, 39, 42, 44, 46, 49, 51, 53, 55, 57 and 59). Instead of drawing the line, the ROM raises Report B, “Integer out of range”. The value of k remains visible on screen, and execution can be resumed with GO TO 20, without resetting n. The replacement routine completes all 60 iterations and terminates normally.

3.8 Differences from the 1982 routine

Table 3.5 summarizes the deliberate differences between the 1982 and new implementations for large and special angles.

Aspect	1982 ROM	New routine
Number of segments	4·INT(|A|·SQR Z/8) + 4	4 + ROUND(|A|·(R’ + 48)/16)
Range	4 to 252, multiples of 4	4 to 252 (never above 255)
Angle reduced modulo 2π	No	Optional: bit 6 of FLAGS2
A = 0 or A = 2πk exactly	Straight line when sin(G/2) = 0 exactly; 21 of the first 60 multiples fail	Straight line for all 60 (tolerance)

Table 3.5 — Behavior of the two routines for large and special angles.